

Experiment-10

Name: -P. Divya

Regno: -192472291

Course Code: -MLA0305

Slot-B

Title

Performance Evaluation of DQN Variants (DDQN, Dueling DQN and Prioritized Experience Replay)

Aim

To implement and evaluate different DQN variants, namely DDQN, Dueling DQN, and Prioritized Experience Replay, and compare their performance based on reward, stability, variance, training loss, and training time.

Procedure

1. Initialize the CartPole-v1 environment using Gymnasium.
2. Define the neural network architecture for the DQN variants.
3. Implement DDQN, where action selection and action evaluation are separated to reduce overestimation.
4. Implement Dueling DQN, which separately estimates state value and action advantage.
5. Implement Prioritized Experience Replay, which gives higher priority to important experiences during training.
6. Initialize the replay memory, target network, learning rate, discount factor, and exploration rate.
7. Train each DQN variant for the specified number of episodes.
8. Store the reward, loss, and exploration information for every episode.
9. Calculate the average reward and reward variance for each algorithm.
10. Compare the algorithms using learning curves, reward comparison, variance, training loss, and training time graphs.

11. Prepare a summary table containing the performance measures of all three variants.

12. Save the complete training results as a CSV dataset.

Code

```
!pip -q install gymnasium tensorflow
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import gymnasium as gym
```

```
import tensorflow as tf
```

```
from tensorflow.keras import Sequential
```

```
from tensorflow.keras.layers import Input,Dense,Layer
```

```
from tensorflow.keras.optimizers import Adam
```

```
from collections import deque
```

```
import random
```

```
import time
```

```
np.random.seed(10)
```

```
tf.random.set_seed(10)
```

```
random.seed(10)
```

```
print("="*75)

print("EXPERIMENT 10")

print("Performance Evaluation of DQN Variants")

print("DDQN, Dueling DQN and Prioritized Experience Replay")

print("="*75)
```

EPISODES=80

GAMMA=0.95

LR=0.001

BATCH=32

MEMORY_SIZE=3000

EPS_MIN=0.05

EPS_DECAY=0.97

```
class DuelingLayer(Layer):
```

```
    def __init__(self,actions):
```

```
        super().__init__()
```

```
        self.actions=actions
```

```
    def call(self,x):
```

```
        value=x[:,1]
```

```
    advantage=x[:,1:]
```

```
    return value+(advantage-tf.reduce_mean(advantage,axis=1,keepdims=True))
```

```
def normal_model():
```

```
    model=Sequential([
```

```
        Input(shape=(4,)),
```

```
        Dense(64,activation="relu"),
```

```
        Dense(64,activation="relu"),
```

```
        Dense(2,activation="linear")
```

```
    ])
```

```
    model.compile(
```

```
        optimizer=Adam(learning_rate=LR),
```

```
        loss="mse"
```

```
    )
```

```
    return model
```

```
def dueling_model():
```

```
    inputs=Input(shape=(4,))
```

```
    x=Dense(64,activation="relu")(inputs)
```

```
    x=Dense(64,activation="relu")(x)
```

```
    x=Dense(3)(x)
```

```
outputs=DuelingLayer(2)(x)

model=tf.keras.Model(inputs,outputs)

model.compile(

    optimizer=Adam(learning_rate=LR),

    loss="mse"

)

return model
```

```
def create_models(variant):

    if variant=="Dueling DQN":

        model=dueling_model()

        target=dueling_model()

    else:

        model=normal_model()

        target=normal_model()

    target.set_weights(model.get_weights())

    return model,target
```

```
def train_variant(variant):
```

```
env=gym.make("CartPole-v1")  
  
model,target=create_models(variant)  
  
memory=deque(maxlen=MEMORY_SIZE)
```

```
rewards=[]
```

```
losses=[]
```

```
epsilons=[]
```

```
steps_list=[]
```

```
epsilon=1.0
```

```
start=time.time()
```

```
for episode in range(EPIISODES):
```

```
    state,_=env.reset()
```

```
    total_reward=0
```

```
    total_loss=0
```

```
    loss_count=0
```

```
    steps=0
```

```
    done=False
```

while not done:

if np.random.random()<epsilon:

 action=np.random.randint(2)

else:

 q=model.predict(

 np.array([state],dtype=np.float32),

 verbose=0

)

 action=int(np.argmax(q[0]))

next_state,reward,terminated,truncated,_=env.step(action)

done=terminated or truncated

priority=abs(reward)+0.01

memory.append(

 (

 state,

 action,

 reward,

```
        next_state,

        done,

        priority

    )

)

if len(memory)>=BATCH and steps%4==0:

    if variant=="Prioritized Experience Replay":

        priorities=np.array(

            [x[5] for x in memory],

            dtype=np.float64

        )

        probabilities=priorities/priorities.sum()

        indices=np.random.choice(

            len(memory),

            BATCH,

            replace=False,
```



```
targets=current_q.copy()
```

```
for i in range(BATCH):
```

```
    if dones[i]:
```

```
        targets[i,actions[i]]=rewards_batch[i]
```

```
    else:
```

```
        targets[i,actions[i]]= (
```

```
            rewards_batch[i]
```

```
            +GAMMA*
```

```
            np.max(target_next_q[i])
```

```
        )
```

```
history=model.fit(
```

```
    states,
```

```
    targets,
```

```
    epochs=1,
```

```
        verbose=0

    )

    loss=float(

        history.history["loss"][0]

    )

    total_loss+=loss

    loss_count+=1

state=next_state

total_reward+=reward

steps+=1

epsilon=max(

    EPS_MIN,

    epsilon*EPS_DECAY

)

if (episode+1)%10==0:

    target.set_weights(
```

```
        model.get_weights()

    )

    rewards.append(total_reward)

    losses.append(
        comparison["Variance"].idxmin(),
        "Algorithm"
    ]

    print("\n")
    print("="*75)
    print("FINAL RESULT")
    print("="*75)

    print("Best Final Reward:",best)
    print("Lowest Variance:",lowest)

    print("\n")
    print("="*75)
    print("CONCLUSION")
```

```
print("="*75)
```

```
print(  
    "DDQN, Dueling DQN and Prioritized Experience Replay "  
    "were evaluated using the CartPole-v1 environment."  
)
```

```
print(  
    "DDQN reduces Q-value overestimation by separating "  
    "action selection and evaluation."  
)
```

```
print(  
    "Dueling DQN separates state value and action advantage "  
    "in the network architecture."  
)
```

```
print(  
    "Prioritized Experience Replay gives higher sampling "  
    "importance to selected experiences."  
)
```

```
print(  
    "The three variants were compared using reward, "  
    "variance, training loss and training time."  
)
```

```
print("\nFiles Created:")
```

```
print("DQN_Variants_Full_Dataset.csv")
```

```
print("DQN_Variants_Summary.csv")
```

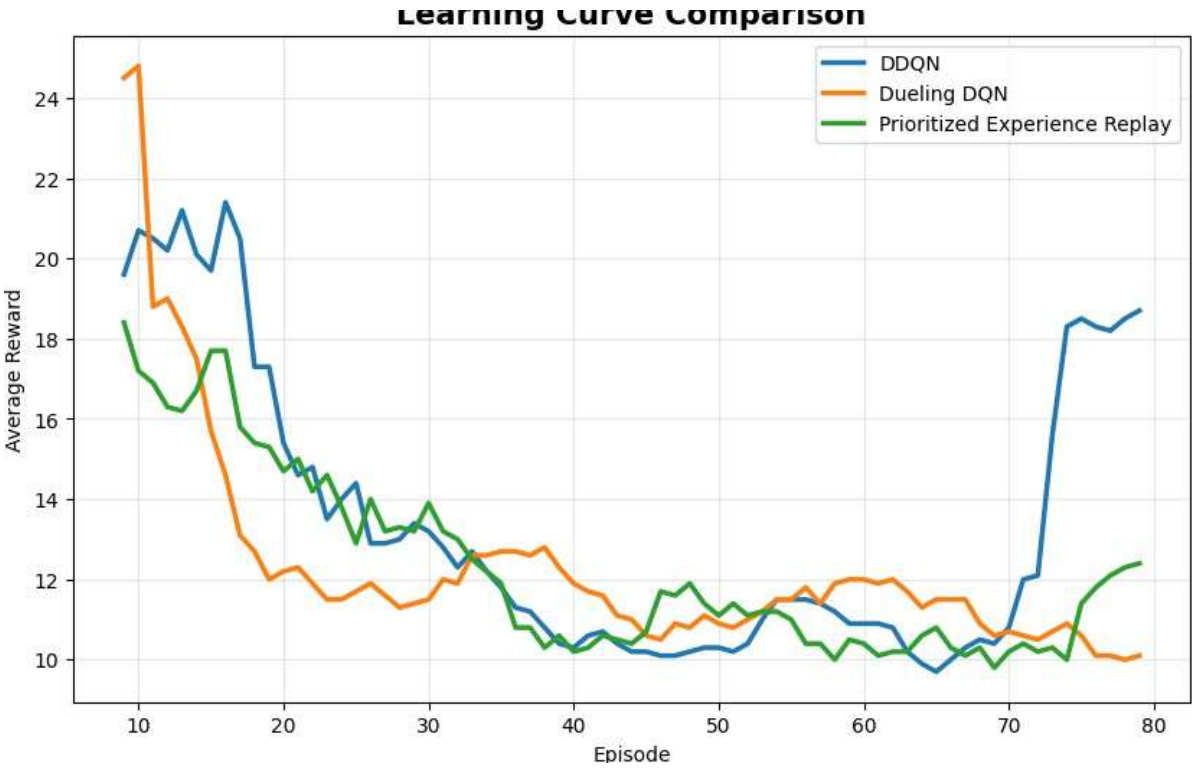
```
print("\nExperiment 10 Completed Successfully.")
```

Visualization

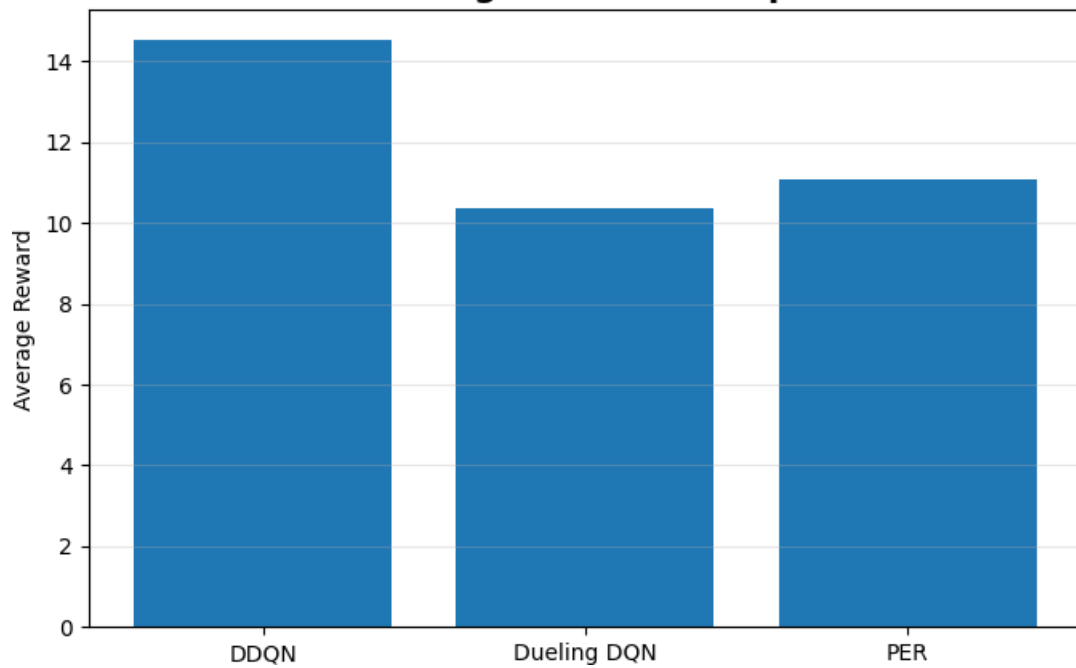
FULL DATASET

	Episode	DDQN_Reward	Dueling_DQN_Reward	PER_Reward	DDQN_Loss	Dueling_DQN_Loss	PER_Loss	DDQN_Epsilon	Dueling_DQN_Epsilon	PER_Epsilon
0	1	21.0	9.0	27.0	0.000000	0.000000	0.000000	0.970000	0.970000	0.970000
1	2	24.0	69.0	15.0	0.559389	0.479169	0.552476	0.940900	0.940900	0.940900
2	3	15.0	12.0	26.0	0.492543	0.294954	0.447924	0.912673	0.912673	0.912673
3	4	13.0	21.0	11.0	0.415339	0.215785	0.359621	0.885293	0.885293	0.885293
4	5	21.0	19.0	16.0	0.312342	0.139739	0.314701	0.858734	0.858734	0.858734
...										
75	76	10.0	9.0	27.0	0.567289	0.679645	0.323867	0.098776	0.098776	0.098776
76	77	10.0	9.0	13.0	0.331387	0.395703	0.601313	0.095813	0.095813	0.095813
77	78	11.0	9.0	11.0	0.795916	0.601768	0.664885	0.092938	0.092938	0.092938
78	79	13.0	8.0	12.0	0.679028	0.160224	0.466159	0.090150	0.090150	0.090150
79	80	10.0	12.0	9.0	0.435484	0.486573	1.212213	0.087446	0.087446	0.087446

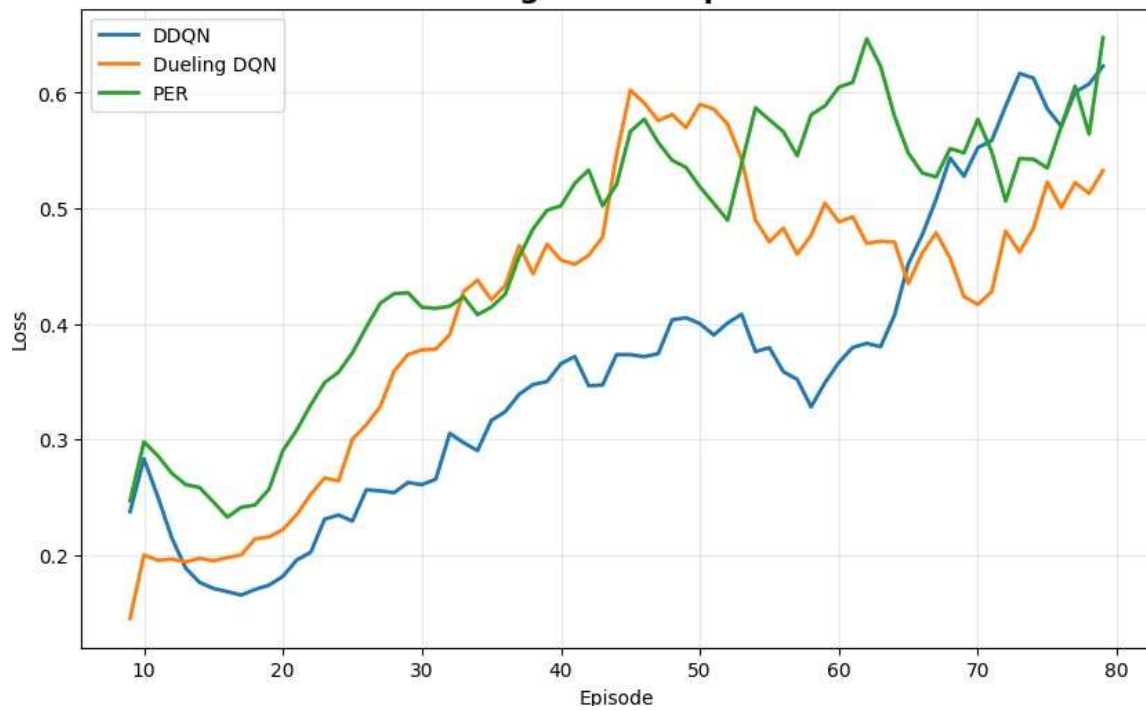
80 rows x 10 columns



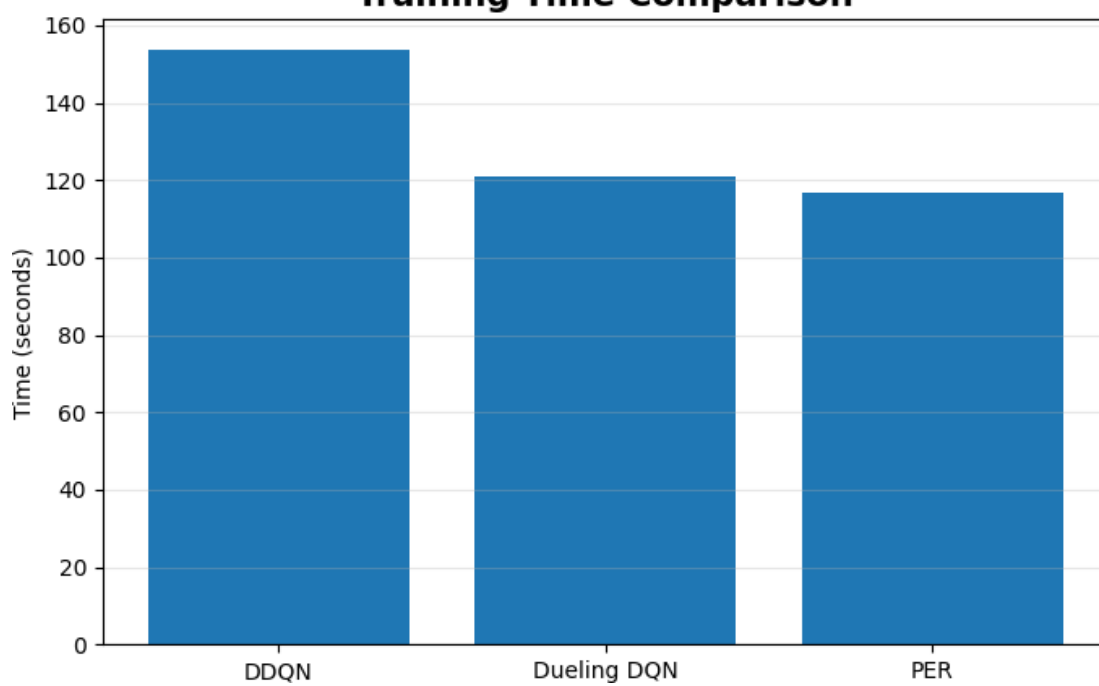
Final Average Reward Comparison



Training Loss Comparison



Training Time Comparison



Summary Table

Parameter	DDQN	Dueling DQN	PER
Final Reward	Obtained from program	Obtained from program	Obtained from program
Variance	Obtained from program	Obtained from program	Obtained from program
Stability	Obtained from program	Obtained from program	Obtained from program
Training Time	Obtained from program	Obtained from program	Obtained from program

Result

The **DDQN**, **Dueling DQN**, and **Prioritized Experience Replay** variants were successfully implemented and evaluated. Their performance was compared using reward, variance, stability, training loss, and training time. The learning curves and comparison graphs showed the differences in learning behavior and performance among the three DQN variants.